ATC Testing

Advanced Trigonometry Calculator Testing

This document explains the current test infrastructure for Advanced Trigonometry Calculator (ATC).

Regression Test Runner

The main automated regression runner is:

```
tests\run-atc-regression.ps1
```

Run it after building ATC:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-regression.ps1 -AtcExe .\x64\Release\atc.exe
```

For x86 builds:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-regression.ps1 -AtcExe .\Release\atc.exe
```

Historical validation recorded before the final 2.1.9 candidate:

These results do not approve the current candidate. Final validation must identify the source snapshot and executable hashes.

```
Summary: 377 passed, 0 failed
```

At that historical checkpoint, Release x86 and Release x64 builds completed with `0 Warning(s), 0 Error(s)`, and both executables passed the same regression suite.

Do not run ATC test runners in parallel against the same ATC data directory. The runners intentionally mutate settings, variables and temporary files under the ATC data folder. Parallel execution is only safe when each ATC process uses an isolated data directory.

Memory Stress Runner

The memory stress runner is:

```
tests\run-atc-memory-stress.ps1
```

Example:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-memory-stress.ps1 -Iterations 10
```

It currently focuses on repeated polynomial simplification, roots-to-polynomial, and textual/coefficient equation solver commands.

SRS Gate Runner

The SRS gate runner is:

```
tests\run-atc-srs-gates.ps1
```

It validates selected measurable requirements from `docs\SOFTWARE_REQUIREMENTS_SPECIFICATION.md`:

- Stage 1: parser and syntax guard matrix with 50+ valid/malformed inputs;
- Stage 2: precision persistence and startup gate;
- Stage 3: degree-20 polynomial simplification / equation solving latency and

Memory Factor telemetry;

- Stage 4: command bridge / TXT detector fixture with mocked external opening

and unrelated-directory preservation.

Quick validation without the heavy stress loop:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-srs-gates.ps1 -AtcExe .\x64\Release\atc.exe -SkipStress
```

Release stress validation:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-srs-gates.ps1 -AtcExe .\x64\Release\atc.exe -StressIterati
```

Latest short SRS gate validation:

```
Summary: 4 passed, 0 failed
StressIterations: 2
```

Isolated Coverage Helper

The isolated coverage helper is:

```
tests\run-atc-isolated-coverage.ps1
```

Use it when a specific area needs focused validation outside the full regression run.

Current isolated coverage result:

```
Summary: 68 passed, 0 failed
```

Current SourceForge package validation result:

```
Summary: 44 passed, 0 failed
```

Current Automated Coverage

The regression suite currently covers:

- arithmetic basics and precedence;
- parser syntax guards for malformed parentheses, empty function arguments, consecutive operators and invalid operator placement;
- constants and precision formatting;
- trigonometry and inverse trigonometry;
- hyperbolic and inverse hyperbolic functions;
- logarithms and custom guide syntax;
- complex-number paths used by documented commands;
- matrix/list functions, matrix helper commands, direct matrix/vector indexing and persisted indexed matrix updates;
- variable names and implicit multiplication;
- sorting and ASCII sorting flows;
- roots-to-polynomial report export;
- polynomial simplification;
- equation solving and systems of equations;
- solver paths including selected trigonometric, hyperbolic, polynomial, and rational-cancellation cases;
- function study examples;
- graph settings, deterministic graph navigation simulation, navigation edge clamping and explicit graph-range navigation;
- date/time commands that can be automated safely;
- TXT detector / command bridge commands, including redirected CMD input, mocked shell/window side effects, and real-file `solve txt` processing with mocked answer-file opening;
- file/folder command existence checks;
- app environment commands that can run safely under redirected input, including `about` and `auto adjust window`;
- variable/result management commands;

- persistent settings and menu retry behavior;

- one safe runtime smoke path for each deep interactive module: unit

conversions, microeconomics, physics, statistics, geometry, financial calculations, triangles rectangles solver, and arithmetic matrix solver;

- verbose-resolution behavior;

- interactive prompt behavior, including Tab autocomplete vocabulary,

ambiguous-match cycling and Up/Down history navigation;

- `double` and Boost `mp_float` precision mode persistence.

The detailed test matrix is maintained in:

`tests\ATC_AUTOMATED_TEST_CASES.md`

Documentation to Test Traceability

Documented behavior should have automated coverage whenever it is deterministic and safe to execute under redirected input.

Use these rules when updating documentation:

- stable command examples should be added to the regression suite where

practical;

- interactive modules should have at least a safe smoke test or an explicit

manual-validation note;

- commands that open windows, files, browsers or PC-control actions should use

mocks, source-level checks or manual validation notes;

- do not update the documented test count unless the suite has actually been

executed;

- update `tests\ATC_USER_GUIDE_COVERAGE.md` when a documented area gains or

loses coverage.

The Cookbook and Best Practices documents are intended to use already documented commands. If a recipe cannot be tested safely, it should say so through notes or limitations.

Known Manual or Interactive Validation Gaps

The automated suite intentionally avoids or only partially covers:

- shutdown, restart, hibernate, sleep, lock, and similar PC-control commands;

- full-duration clock, stopwatch, timer, and continuously running views;

- commands that open external browsers or editors, except for TXT/file flows

that now have explicit mock coverage;

- full live manual graph rendering and key-buffer behavior;

- remaining branches inside deeply interactive modules such as triangles,

arithmetic matrix solver, conversions, finance, physics, microeconomics, and statistics menus.

Long-running time command branches, short stopwatch marking, zero-duration clock rendering, and timer syntax guards are covered by the isolated helper. Full-duration countdowns, alarms, and continuously running views should still be validated manually or covered later with safe, non-destructive automation.

The direct TXT workflow is covered with temporary fixtures: `predefine txt` writes a real input path, `solve txt` generates the `_answers.txt` response file, invalid lines are kept isolated from later lines, and the answer-file open is recorded through the test mock instead of launching Notepad. The fixture also covers `solve equation` and `simplify polynomial` inside the TXT input, including their export-prompt-safe non-interactive path. `auto solve txt` now has a deterministic runtime fixture that uses a real TXT file containing `SOLVE_NOW`, verifies that the flag is consumed, validates

the generated answer file, and records the answer-file open through the test mock.

SourceForge Package Validation

The package validation runner is:

```
tests\run-atc-package-validation.ps1
```

It validates the staged SourceForge package structure without rebuilding the ZIP. Current checks include root files, x64/x86 runtime executables, generated PDF documentation, absence of duplicated source folders, absence of generated answer files, GitHub source notices, and SHA256 checksum consistency for every packaged file.

Example:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-package-validation.ps1
```

Settings Modified by Tests

The regression runner may temporarily touch ATC setting files under:

```
%USERPROFILE%\Pictures\Advanced Trigonometry Calculator
```

The runner backs up and restores the settings it modifies.

Recommended Validation Before Changes

For most changes:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-regression.ps1 -AtcExe .\x64\Release\atc.exe
```

For memory-sensitive work, also run the memory stress runner and the script benchmark when practical.

Script Performance Validation

The script interpreter has additional automated benchmark coverage for the `Scripts examples/Multiplication Table 1-100_script_generator.txt` workflow. This scenario is useful because it repeatedly exercises:

- nested `for` loops;
- variable assignment and reuse;
- `print("...", ...)` formatting;
- repeated scalar expression evaluation inside script output.

ATC 2.1.7 reduces memory pressure in this flow by releasing temporary TXT-processing arrays, avoiding matrix scratch allocation for scalar expressions, executing safe `print("...", ...)` script lines directly in memory instead of routing them through the temporary TXT-processing file path, and evaluating simple scalar assignments, loop conditions and integer `print` arguments through a lightweight script path.

Run the benchmark with:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-script-benchmark.ps1 -AtcExe .\x64\Release\atc.exe
```

The benchmark validates process exit, elapsed time, peak working set and stable output content. Historical Release x64 validation:

```
Summary: 4 passed, 0 failed
ElapsedSeconds: 7.41
PeakWorkingSetMB: 119.56
```